

REACT JS BASED WEB APPLICATION FOR TICKET BOOKING OF AN INTERNAL DEPARTMENT EVENT

School of Computing

**Bachelor of Technology
in
Computer Science & Engineering**

By

T.INDRANI(VTU24820)

10211CS224

Full Stack Application Development

Slot : E2-B20



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D
INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA**

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled " React JS based web application for Ticket Book-ing of an Internal Department Event" by T.INDRANI(VTU24820) has been carried out in Winter semester of Academic year 2025-2026(WS2526).

Signature of Faculty

Dr.S.Hemamalini

Assistant professor

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

Signature of Course Co-Ordinator

Dr.Sumithra.M

Assistant Professor(Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

T.INDRANI

Date:06/05/2026

ABSTRACT

The proposed system is a React JS-based web application designed to streamline the process of ticket booking for internal department events such as technical fests, workshops, and seminars. Traditional ticket booking methods often involve manual coordination, fragmented communication, and lack of real-time updates, leading to inefficiencies and user inconvenience. This application addresses these challenges by providing a centralized, user-friendly platform that enables seamless event discovery, ticket reservation, and confirmation.

The system leverages React JS for building a dynamic and responsive user interface, ensuring a smooth user experience across devices. It integrates features such as event listing, user registration and authentication, seat selection, ticket generation, and booking history management. Additionally, the application supports real-time data handling and state management to ensure accurate ticket availability and prevent overbooking.

The platform can be extended with secure payment gateway integration and QR-based ticket verification for event entry, enhancing both usability and security. By digitizing the entire booking workflow, the

system minimizes manual errors, reduces administrative overhead, and improves operational efficiency for event organizers.

Overall, the application provides a scalable and efficient solution for managing internal event ticketing, offering convenience to users and valuable insights to organizers through booking analytics.

Keywords: React JS, Ticket Booking System, Web Application, Event Management, User Interface, Real-time Data Processing, Online Reservation System, QR Code Verification, Frontend Development, Digital Ticketing.

LIST OF FIGURES

1.1	Framework of the Ticket Booking Application	3
3.1	Booking Records Stored in MySQL Database	8
1.	Proposed System Architecture of Ticket Booking Ap- plication.....	11
1.	User Interface of Ticket Booking System	14
2.	Booking Confirmation Output.....	15
3.	Booking Records Stored in MySQL	15

LIST OF TABLES

1.	System Performance and Feature Analysis	15
2.	Mapping of Project to Complex Engineering Problem (CEP).....	18
3.	Mapping of Project to Complex Engineering Activities (CEA)	19
4.	SDG Mapping for the Project.....	20

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
UI	User Interface
UX	User Experience
DB	Database
SQL	Structured Query Language
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
CI	Continuous Integration
ORM	Object Relational Mapping

TABLE OF CONTENTS

	Page.No
ABSTRACT	iii
LIST OF FIGURES	v
LIST OF TABLES	vi
LIST OF ACRONYMS AND ABBREVIATIONS	vii
1 INTRODUCTION	1
1. Introduction	1
2. Aim of the Project	1
3. Scope of the Project	2
4. Framework	2
2 SURVEY OF SIMILAR APPLICATION IN MARKET	4
3 FRAMEWORK DESCRIPTION	6
1. Frontend Implementation	6

2.	Backend Implementation	7
3.	Database Implementation	8
4	METHODOLOGIES	10
1.	Project Architecture.....	10
2.	DataFlow Diagram	11
3.	Module 1: User Interface Module	12
4.	Module 2: API and Backend Module.....	12
5.	Module 3: Database Module	12
5	RESULTS AND DISCUSSIONS	14
6	CONCLUSION AND FUTURE ENHANCEMENTS	16
1.	Conclusion.....	16
2.	Future Enhancements	17
7	ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG	18
1.	Mapping of the Project to Complex Engineering Ac- tivities(CEA)	18
2.	SDG Mapping	20

8	References	21
9	SOURCE CODE	22
	1. Source Code	26

Chapter 1

INTRODUCTION

1. Introduction

Managing ticket bookings for internal department events through manual methods is often inefficient and prone to errors. To address this, a React JS-based web application is developed to provide a simple and organized platform for event registration and ticket booking. The system enables users to view events, register, and book tickets easily, while helping organizers manage data efficiently. By leveraging React JS, the application ensures a responsive interface and smooth user experience, making the overall process faster, more reliable, and user-friendly.

2. Aim of the Project

The aim of this project is to design and develop a React JS-based web application for efficient ticket booking of internal department events.

The system focuses on simplifying the process of event registration and ticket management by providing a user-friendly interface, ensuring accurate data handling, and reducing manual effort for both users and organizers.

3. Scope of the Project

The scope of this project includes the development of a React JS-based web application for managing ticket bookings of internal department events such as seminars, workshops, and technical fests. The system allows users to view event details, register, and book tickets, while enabling organizers to manage event information and participant records efficiently. The application focuses on providing a responsive user interface, real-time updates, and accurate booking management. Future enhancements may include payment gateway integration, QR code-based ticket verification, and analytics for event performance.

4. Framework

The proposed ticket booking application is developed using the React JS framework, a powerful JavaScript library for building dynamic and responsive user interfaces. React enables a component-based architecture, allowing the application to be divided into reusable and

manageable components such as event listings, booking forms, and user dashboards. It utilizes a virtual DOM to efficiently update and render changes, ensuring high performance and a smooth user experience. Additionally, React integrates well with backend services and APIs, making it suitable for handling real-time booking data, user authentication, and event management functionalities within the system.

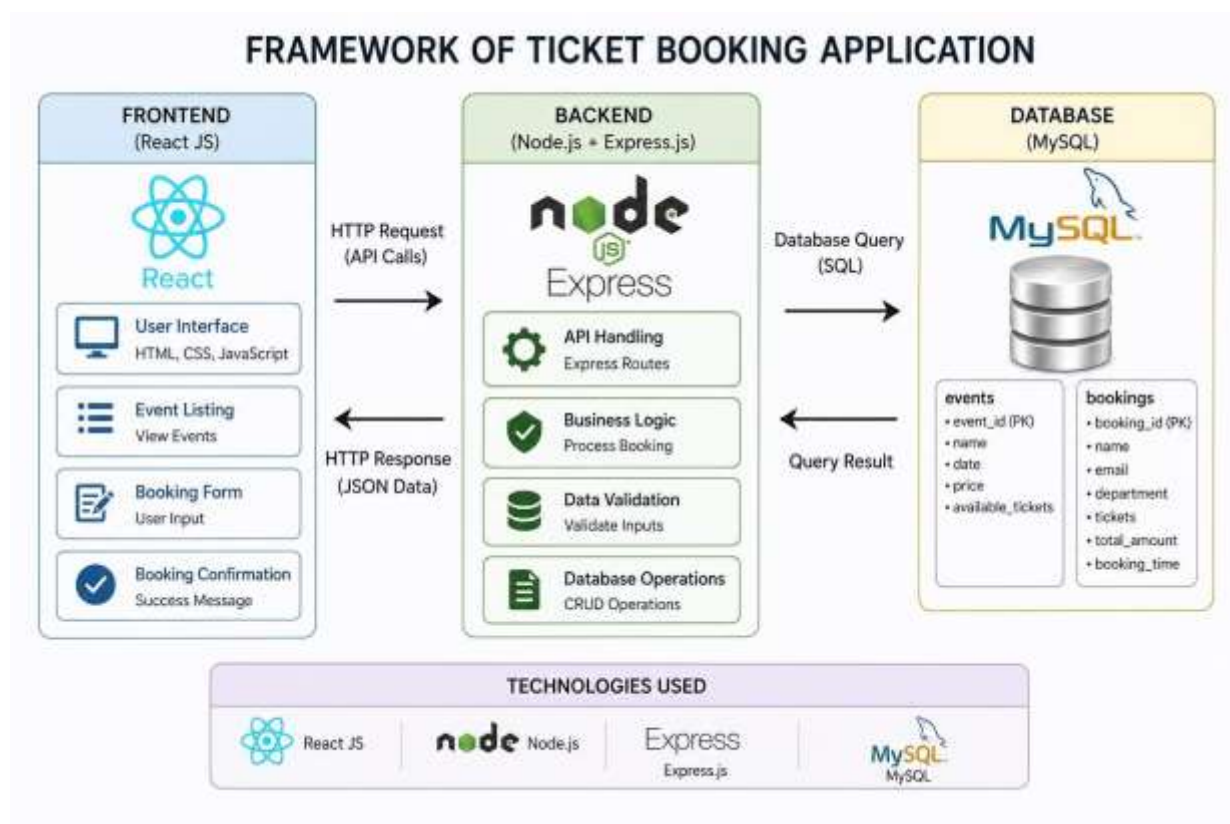


Figure 1.1: Framework of the Ticket Booking Application

This diagram shows the overall structure of the ticket booking system with three layers: frontend, backend, and database. The frontend sends user requests to the backend, which processes them and interacts with the database. Finally, the response is sent back to the frontend for display.

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

In the current market, several online ticket booking platforms have been developed to simplify event discovery and reservation processes. One of the most prominent platforms is BookMyShow, which provides services for booking tickets for movies, concerts, sports events, and other live shows across India. It offers features such as event listings, seat selection, secure payment options, and user-friendly navigation, making it widely popular among users [ref1]. Similarly, global platforms like Ticketmaster and StubHub provide large-scale ticketing solutions with secure transactions, event promotion, and customer support, catering to millions of users worldwide [ref2].

Most modern ticket booking applications share common features including user registration, event search and filtering, real-time ticket availability, online payments, and booking history management. Ad-

vanced systems also provide personalized recommendations, analytics dashboards for organizers, and fraud detection mechanisms to ensure secure transactions [ref3]. Despite their efficiency, some existing systems face limitations such as high service charges, system overload during peak demand, and refund-related issues.

This survey highlights that while current applications are highly functional and scalable, there is still scope for developing simplified and cost-effective solutions tailored for smaller environments such as internal department events. The proposed React JS-based application focuses on addressing these needs by providing a lightweight, user-friendly, and efficient ticket booking system specifically designed for academic institutions.

Chapter 3

FRAMEWORK DESCRIPTION

1. Frontend Implementation

1. Environment Setup

The development environment is set up using Node.js and npm for managing dependencies. The frontend is built using React JS along with HTML and CSS for designing the user interface. The backend data is managed using MySQL, and commands are executed through the command line interface. Development is carried out in Visual Studio Code, with version control handled using Git and GitHub. The application is tested using a web browser such as Google Chrome.

2. Structure of the Project

The project follows a modular structure with separate components for better organization and maintainability. The frontend is developed using React JS, divided into reusable components such as event display, booking form, and user interface elements. Styling is handled

using CSS. The backend uses MySQL to store event details, user data, and booking information. The project is organized into folders for components, assets, and database scripts to ensure clarity and scalability.

3.1.3 Execution Procedure

To run the application, the required dependencies are first installed using npm. The React application is started using the appropriate command in the terminal. The MySQL database is configured and connected to store and retrieve booking data. Once the server is running, the application can be accessed through a web browser, allowing users to view events and book tickets efficiently.

2. Backend Implementation

1. Environment Setup

The backend environment is configured using MySQL as the database system. The database is managed through the command line interface, where tables for events and bookings are created and maintained. A local server setup (Node.js-based API) is used to handle communication between the frontend and the database.

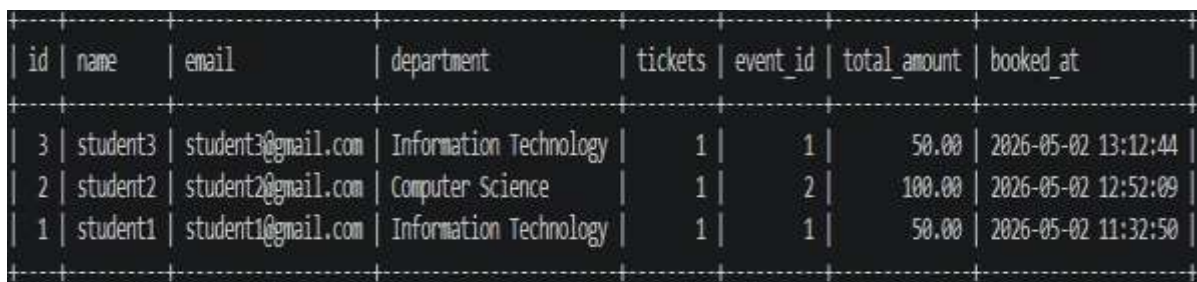
2. Structure of the Project

The backend follows a simple API-based structure. It includes end-

points such as /api/event to fetch event details and /api/book to handle ticket bookings. The MySQL database contains tables for storing event information (name, date, price, available tickets) and booking details (user name, email, department, and number of tickets). The frontend interacts with these endpoints using HTTP requests.

3.2.3 Execution Procedure

The backend is executed by first starting the MySQL service and selecting the required database. The necessary tables such as bookings are created and managed using SQL commands through the command line interface. The Node.js server is then started using the command `node server.js`. Once the server is active, the frontend sends API requests to fetch event details and store booking information. The booking data is successfully stored in the MySQL database and can be retrieved using queries such as `SELECT * FROM bookings;`

A screenshot of a MySQL database table named 'bookings'. The table has 8 columns: id, name, email, department, tickets, event_id, total_amount, and booked_at. There are three rows of data. The first row shows a booking by student3 for event 1, with a total amount of 50.00, booked at 2026-05-02 13:12:44. The second row shows a booking by student2 for event 2, with a total amount of 100.00, booked at 2026-05-02 12:52:09. The third row shows a booking by student1 for event 1, with a total amount of 50.00, booked at 2026-05-02 11:32:50.

id	name	email	department	tickets	event_id	total_amount	booked_at
3	student3	student3@gmail.com	Information Technology	1	1	50.00	2026-05-02 13:12:44
2	student2	student2@gmail.com	Computer Science	1	2	100.00	2026-05-02 12:52:09
1	student1	student1@gmail.com	Information Technology	1	1	50.00	2026-05-02 11:32:50

Figure 3.1: Booking Records Stored in MySQL Database

This image shows the booking records stored in the MySQL database. It includes details like user name, email, department, tickets, event ID, total amount, and booking time.

3. Database Implementation

1. Database Configuration

The database is configured using MySQL to store and manage event and booking data. A database named `techspark_db` is created and selected using SQL commands. The configuration is performed through the MySQL command line interface, ensuring proper setup for data storage and retrieval operations required by the application.

2. Schema Structure

The database schema is designed to efficiently handle event and booking information. It includes structured tables with appropriate fields such as event details (name, date, price, available tickets) and booking details (user name, email, department, number of tickets, total amount, and timestamp). Relationships are maintained using identifiers like `event_id` to link bookings with the corresponding event.

3. Tables Created

The main tables created in the database include the events table and the bookings table. The events table stores information about the event such as event name, date, venue, total tickets, and available tickets. The bookings table stores user booking details including name, email, department, number of tickets booked, total amount, and booking time. These tables enable efficient storage and retrieval.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The project follows a layered architecture consisting of presentation, application, and database layers. The presentation layer includes the frontend developed using HTML, CSS, and React JS, which allows users to view event details and book tickets. The application layer consists of a Node.js backend server that handles API requests, processes booking data, and manages business logic. The database layer uses MySQL to store event and booking information.

The frontend communicates with the backend through API requests, and the backend interacts with the database to perform data storage and retrieval. Additional components such as authentication and deployment are considered in the architecture design for future enhancements.

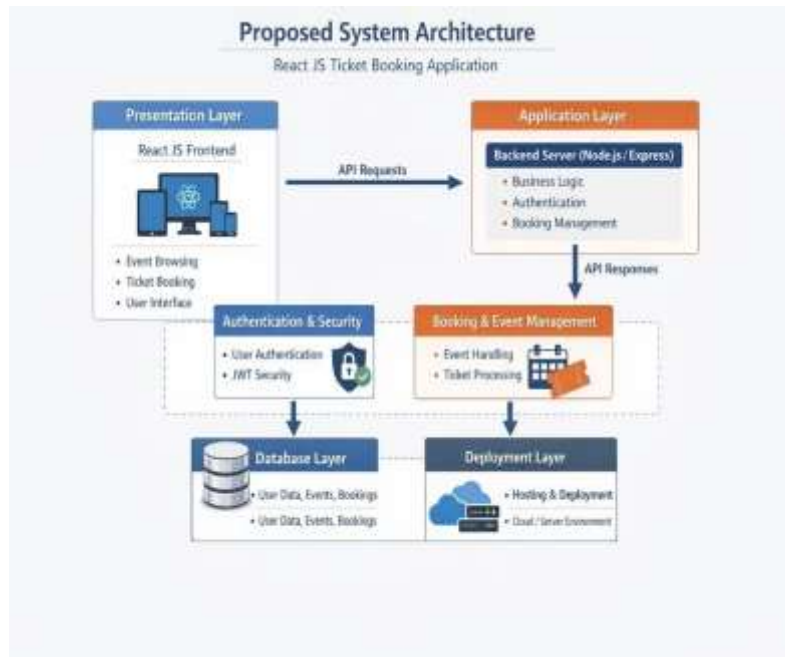


Figure 4.1: Proposed System Architecture of Ticket Booking Application

This image shows the proposed system architecture of the ticket booking application with presentation, application, and database layers. It illustrates how user requests flow from frontend to backend, where processing and authentication happen, and data is stored in the database.

4.2 DataFlow Diagram

The data flow begins when the user accesses the web application and views event details. The frontend sends a request to the backend API to fetch event information. When a user fills the booking form and submits it, the data is validated and sent to the backend server. The backend processes the request, stores the booking details in the MySQL database, and updates ticket availability. The response is then sent back to the frontend, which displays the booking confirmation to the user.

3. Module 1: User Interface Module

CampusTicketPro is a full-stack campus ticket booking system built with React/Vite frontend, Node.js Express backend and MySQL. The backend exposes auth, event, ticket, attendance, and admin stats endpoints. The database schema supports users, events, tickets, registrations, and attendance. The root package scripts start both frontend and backend together for development. The frontend UI lives in App.jsx and is mounted by main.jsx. It includes a hero landing section, feature highlights, and a live updates ticker. A navigation bar provides smooth in-page movement to Home, Features, About, and Contact.

4. Module 2: API and Backend Module

This module manages communication between the frontend and the database. It includes API endpoints such as fetching event details and processing ticket bookings. The backend validates input data, handles business logic, and ensures proper data flow between the frontend and MySQL database. The backend module is built with Express in src, exposing REST API routes for auth, events, tickets, attendance, and admin stats. The API controllers validate input, call MySQL models, and return JSON responses for the frontend.

5. Module 3: Database Module

This module is responsible for data storage and management. It includes the MySQL database with tables such as events and bookings. It ensures data integrity, efficient storage, and quick retrieval of booking and event information. The database module uses MySQL and is defined in schema.sql plus backend DB model files under models. It stores users, events, tickets, registrations, and attendance records for the campus ticket system. The backend uses mysql2 with a connection pool and model functions to query and update the database. The schema supports role-based users, event metadata, ticket lifecycle, and attendance tracking.

Advantages of this Project

This project provides a simple and efficient solution for managing internal event ticket bookings digitally.

a) User-Friendly Interface: The application provides an easy-to-use interface for users to view events and book tickets quickly.

b) Real-Time Data Management: Ticket availability is updated dynamically, ensuring accurate booking information.

c) Organized Data Storage: All booking and event details are stored systematically in a database, making management easier.

Disadvantages

The project has certain limitations that can be improved in future versions. It currently depends on a local server setup, which limits accessibility. Security features such as authentication and payment integration are not fully implemented. Additionally, the system is designed for small-scale internal use and may require optimization for larger deployments.

Chapter 5

RESULTS AND DISCUSSIONS

The developed ticket booking system was successfully implemented and tested. The application allows users to view event details, check ticket availability, and book tickets efficiently. The frontend provides a user-friendly interface, while the backend processes booking requests and stores data in the MySQL database. The system was tested with multiple inputs, and the booking details were correctly stored and retrieved from the database.

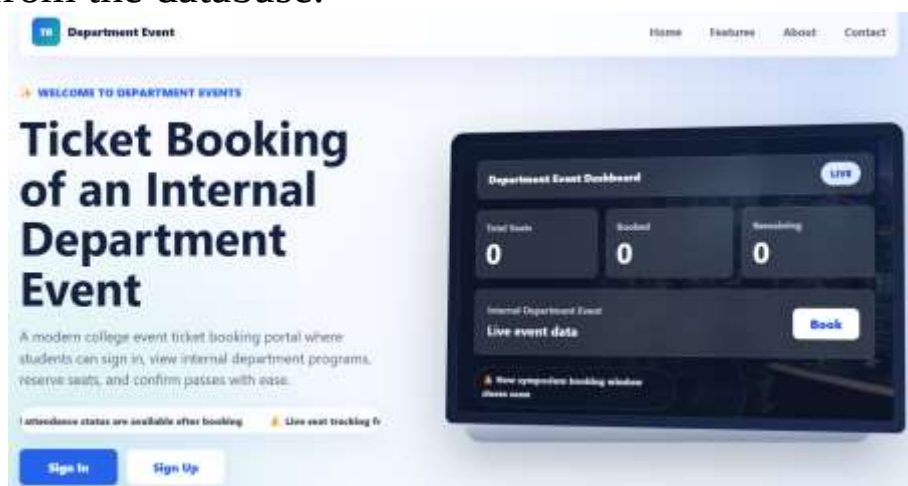


Figure 5.1: User Interface of Ticket Booking System

This image shows the user interface of the ticket booking system where users can browse events and book tickets. It represents the frontend design that allows easy interaction for this is the opening page here student ,faculty and admin can create account and login

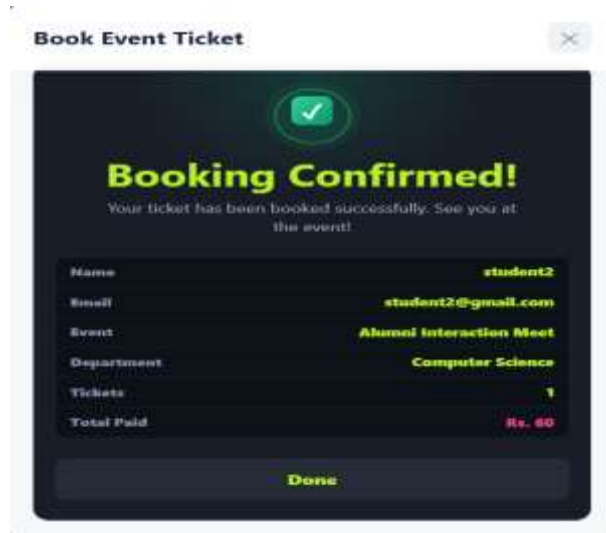


Figure 5.2: Booking Confirmation Output

This image shows the booking confirmation screen displayed after a successful ticket booking ,payment summary.

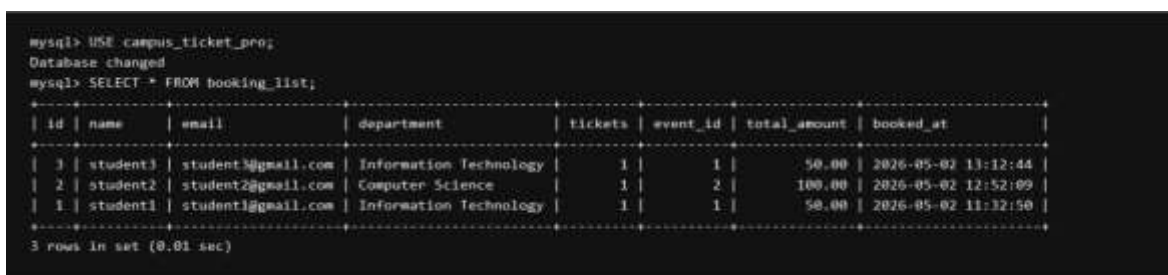


Figure 5.3: Booking Records Stored in MySQL

This image shows the booking records stored in the MySQL database, including user details, event ID, tickets, and total amount.

successful booking.

Table 5.1: System Performance and Feature Analysis

Module	Feature	Observation / Result
Frontend	User Interface	Responsive design with smooth navigation
Frontend	Form Validation	Prevents invalid inputs and ensures accuracy
Backend	Booking Process	Successfully processes and stores bookings
Database	Data Storage	Efficient storage using MySQL tables
System	Error Handling	Displays appropriate error and success messages
System	Performance	Fast response for booking and data retrieval

This table shows the system performance and feature analysis of the ticket booking application. It highlights key modules, their features, and the observed results such as smooth UI, accurate validation, efficient storage, and fast performance.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The ticket booking web application for internal department events was successfully designed and developed using HTML, CSS, React JS concepts, Node.js, and MySQL. The system provides an efficient platform for users to view event details and book tickets through a simple and interactive interface.

The application ensures proper validation of user inputs, accurate calculation of ticket amounts, and reliable storage of booking data in the database. The integration between frontend and backend enables smooth communication and real-time updates. Overall, the project achieves its objective of simplifying the ticket booking process and reducing manual effort.

2. Future Enhancements

The current system can be further improved by adding advanced features and scalability options. Some possible enhancements include:

- Implementing user authentication and login system for secure access
- Integrating online payment gateways for real-time payment processing
- Adding email/SMS notifications for booking confirmation
- Developing an admin dashboard for event and booking management
- Deploying the application on cloud platforms for wider accessibility
- Enhancing UI/UX design for better user experience

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

Table 7.1: Mapping of Project to Complex Engineering Problem (CEP)

Level	Attribute	Characteristics	WKs	Justification
WP1	Depth of Knowledge	Requires in-depth engineering knowledge	WK3, WK4, WK5, WK6	Requires Flask, databases, cybersecurity (OTP, SHA256), AI models (LLM, CNN) for secure system.
WP2	Conflicting Requirements	Technical and non-technical conflicts	WK4, WK5, WK6	Balances security and usability with fast transactions and secure authentication.
WP3	Depth of Analysis	Analytical and design thinking	WK3, WK4, WK5	System design and AI integration for pharmacy operations.
WP4	Familiarity	Partially new problems	WK4, WK8	AI chatbot integration introduces modern challenges.
WP5	Applicable Codes	Limited standard solutions	WK6	Custom AI + OTP security design not in standard systems.
WP6	Stakeholder Needs	Multiple stakeholders involved	WK5, WK6, WK7	Supports staff, admin, and customers securely.
WP7	Interdependence	System-level integration	WK3, WK5, WK6	Integrates frontend, backend, AI, and security layers.

This table shows how the project addresses complex engineering problems by mapping different attributes like knowledge, analysis, and stakeholder needs.

8.1 Mapping of the Project to Complex Engineering Activities(CEA)

Table 7.2: Mapping of Project to Complex Engineering Activities (CEA)

EA No.	Attribute	Characteristics	Justification
EA1	Range of Resources	Use of diverse technologies	Flask, SQLite, AI models, OTP APIs, frontend tech.
EA2	Level of Interactions	Complex module interactions	Billing, AI chatbot, loyalty, security integration.
EA3	Innovation	Modern technologies	AI chatbot + image recognition + OTP security.
EA4	Societal Impact	Impact on users	Improves healthcare access and reduces errors.
EA5	Familiarity	Beyond standard practice	AI + pharmacy system integration.

This table shows the mapping of the project to complex engineering activities, highlighting attributes like technology usage, interactions, and innovation. It explains how the system uses modern technologies and integrations to handle real-world challenges effectively.

8.2 SDG Mapping

Table 7.3: SDG Mapping for the Project

SDG No.	SDG Title	Relevance to the Project
SDG 3	Good Health and Well-being	AI-based healthcare recommendations and safety improvements.
SDG 9	Industry, Innovation and Infrastructure	Digital transformation using AI and secure systems.
SDG 16	Peace, Justice and Strong Institutions	Secure transactions using OTP and encryption.
SDG 12	Responsible Consumption and Production	Reduces medication errors and improves efficiency.

This table shows the SDG mapping of the project, highlighting how it contributes to goals like innovation, security, and efficiency. It explains the relevance of the system in improving digital processes and ensuring secure operations.

References

- 1 Meta Platforms Inc. (2024). React: A JavaScript Library for Building User Interfaces, React Official Documentation, 1(1), pp. 1-10.
- 2 OpenJS Foundation (2024). Node.js Runtime Environment Documentation, Node.js Official Docs, 2(1), pp. 1-12.
- 3 Oracle Corporation (2024). MySQL Relational Database Management System, MySQL Official Documentation, 3(2), pp. 1-15.
- 4 BookMyShow Team (2023). Online Ticket Booking System Architecture and Implementation, BookMyShow Technical Report, 5(4), pp. 45-52.
- 5 Eventbrite Inc. (2023). Event Management and Ticketing Platform System Design, Eventbrite Engineering Journal, 4(3), pp. 30-38.

SOURCE CODE

Source Code

```
1
2 <!DOCTYPE html>
3 <html lang="en">
4 <head>
5 <meta charset="UTF-8" />
6 <meta name="viewport" content="width=device-width , initial-scale=1.0" />
7 <title>TechSpark      Ticket Booking</title>
8
9 <!DOCTYPE html>
10 <html lang="en">
11 <head>
12 <meta charset="UTF-8" />Spark      Ticket Booking</title>
13
14 *, *::before , *::after { box-sizing: border-box; margin: 0; padding: 0; }
15
16 :root {
17   --bg: #162975;
18   --surface: #13141a;
19   --card: #1b1d27;
20   --border: #2a2d3a;
21   --accent: #c81136;
22   --accent2: #e05b3a;
23   --text: #bee417;
24   --muted: #7a7e99;
```

```

25     --success: #2ecc8a;
26     --error: #ff5e6c;
27     --radius: 16px;
28
29 }
30
31 header {
32     background: var(--surface);
33     border-bottom: 1px solid var(--border);
34     padding: 18px 40px;
35     display: flex;
36     align-items: center;
37     justify-content: space-between;
38     position: sticky;
39     top: 0;
40     z-index: 100;
41     backdrop-filter: blur(12px);
42
43 </label>
44     <input type="text" id="inp-name" placeholder="e.g. Ravi Kumar" oninput="
45         handleInput('name', this.value)" />
46     <div class="error-msg" id="err-name" style="display:none"></div>
47
48 <div class="form-group">
49     <label for="inp-email">Email Address </label>
50     oninput="handleInput('email', this.value)" />
51     <div class="error-msg" id="err-email" style="display:none"></div>
52 </div>
53
54 <div class="form-group">
55     <label for="inp-department">Department </label>
56     <select id="inp-department" onchange="handleInput('department', this.
57         value)">
58         <option value="">Select your department</option>
59         <option value="CSE">CSE</option>
60         <option value="ECE">ECE</option>
61         <option value="EEE">EEE</option>
62         <option value="Mechanical">Mechanical</option>

```

```

62     <option value="Civil">Civil</option>
63     <option value="IT">IT</option>
64     <option value="MCA">MCA</option>
65     <option value="MBA">MBA</option>
66 </select>
67 <div class="error -msg" id="err -department" style="display : none"></div>
68 </div>
69
70 <div class="form -group">
71     <label for="inp -tickets">Number of Tickets</label>
72 </div>
73 </div>
74
75 <div class="amount -preview">
76     <span>Total Amount</span>
77     <strong id="totalAmount">          </strong>
78 </div>
79
80 <button class="btn -primary" id="submit Btn" onclick="handleSubmit()">
81     Confirm Booking
82 </button>
83 </div>
84 </div>
85
86 <!-- RIGHT COLUMN -->
87 <div>
88     <div class="card" style="margin -bottom :20px">
89         <div class="card -title"><div class="icon">          </div> Important
90
91     </ul>
92 </div>
93 <div class="card" style="background : linear -gradient (135 deg , rgba
    (244,196,48,0.08) , rgba (224,91,58,0.08)) ;border -color : rgba
    (244,196,48,0.25)">
94     <div class="card -title"><div class="icon">          </div> Contact & Support</
    div>
95     <div style="font -size :0.88rem; color : var (— muted ) ;line -height :1.7">
96     accent)">techspark@cse . edu . in </span ></p>

```

```

97         <p style="margin-top: 12px">         <strong style="color: var(--text)">
          Organiser </strong><br/>CSE Student Council , Block A</p>
98     </div>
99 </div>
100 </div>
101 </div>
102
103 <!-- SUCCESS MODAL -->
104 <div class="modal-overlay" id="modal" style="display: none" onclick="if (event.
    target === this) closeModal ()">
105     <div class="modal">
106         <div class="success-icon">         </div>
107         <h2>Booking Confirmed !</h2>
108         <p>Your ticket has been booked successfully. See you at the event!</p>
109         <div class="summary-box" id="modalSummary"></div>
110         <button class="btn-close" onclick="closeModal ()">Close & Book More</button>
111     </div>
112 </div>
113
114 <script>
115     const API = 'http://localhost:5000/api';
116
117     //         State
118     let eventData = null;
119     let usingMock = false;
120     let form = { name: '', email: '', department: '', tickets: '' };
121
122     const MOCKEVENT = {
123         name: "Tech Spark 2025         National Technical Festival",
124         department: "Department of Computer Science & Engineering",
125         date: "2025-04-25",
126         time: "09:00 AM         06:00 PM",
127         venue: "Main Auditorium , Block A, Ground Floor",
128         price: 199,
129         total_tickets: 200,
130         available_tickets: 200
131     };
132     }
133     renderEventCard();

```

```

134 }
135 <div class="progress-track"><div class="progress-fill" style="width : ${pct}%"
    ></div></div>
136 </div>
137 ‘
138 const ticketCount = parseInt(form.tickets);
139
140 // Mock mode (no backend)
141 if (usingMock) {
142     await new Promise(r => setTimeout(r, 900));
143     const booking = {
144         name: form.name.trim(),
145         email: form.email.trim(),
146         department: form.department,
147         tickets: ticketCount,
148         eventName: eventData.name,
149         totalAmount: ticketCount * eve
            name: form.name.trim(),
150         email: form.email.trim(),
151         department: form.department,
152         tickets: ticketCount
153     });
154     });
155     serverError').innerHTML =
156
157
158     }
159     lay = 'flex';
160 }
161 function closeModal() {
162     document.getElementById('modal').style.display = 'none';
163 }
164
165 loadEvent();
166 </script>
167 </body>
168 </html>

```